

Verification Academy (/)

Search form

Topics

Courses

Forums

Patterns Library

Cookbooks

Events

More

[ISO-26262-faults-straight](#)[Registe](#)[Home \(/\)](#) / [Topics \(https://verificationacademy.com/topics/\)](https://verificationacademy.com/topics/) / [Functional Safety \(https://verificationacademy.com/topics/functional-safety/\)](https://verificationacademy.com/topics/functional-safety/) /

Getting ISO 26262 Faults Straight

Getting ISO 26262 Faults Straight

by [Avidan Efody \(https://verificationacademy.com/users/avidan-efody/\)](https://verificationacademy.com/users/avidan-efody/), Verification Architect, Mentor Graphics.*(If you're looking for an executive summary of ISO 26262 random hardware fault analysis, check out the following [blog post \(https://blogs.mentor.com/verificationhorizons/blog/2015/10/26/getting-iso-26262-faults-straight/\)](https://blogs.mentor.com/verificationhorizons/blog/2015/10/26/getting-iso-26262-faults-straight/).)*

ISO 26262 for automotive requires that the impacts of random hardware faults on hardware used in vehicles are thoroughly analyzed and the risk of safety critical failures due to such faults is shown to be below a certain threshold. For most hardware design and verification engineers this requirement is a passage into a strange world with which they're not familiar: a world where logic could get corrupted anywhere and in any time, where results are probabilistic rather than absolute, and where common analysis techniques such as checkers and coverage need to be re-thought to be effective. In this article, the first of a series, we will explain where random hardware faults are coming from, how the probability for their occurrence is calculated, and how ISO 26262 requires that they're classified. In upcoming posts we will look at how various advanced verification techniques could refine the impact analysis process and help in calculating ISO 26262 metrics to assess the efficiency of various safety mechanisms implemented in hardware.

Random hardware faults – i.e. individual gates going nuts and driving a value they're not supposed to drive – are practically expected in every electronic device, at a very low probability. When we talk about mobile or home entertainment devices, we could well live with their impact. But when we talk about safety critical designs, such as automotive, aerospace or medical, we could well die from it. That is why hardware design and verification engineers working on applications for these domains find themselves having to prove not only correct functionality of normally operating designs, but also safe functionality of designs plagued by random faults. Don't envy them.

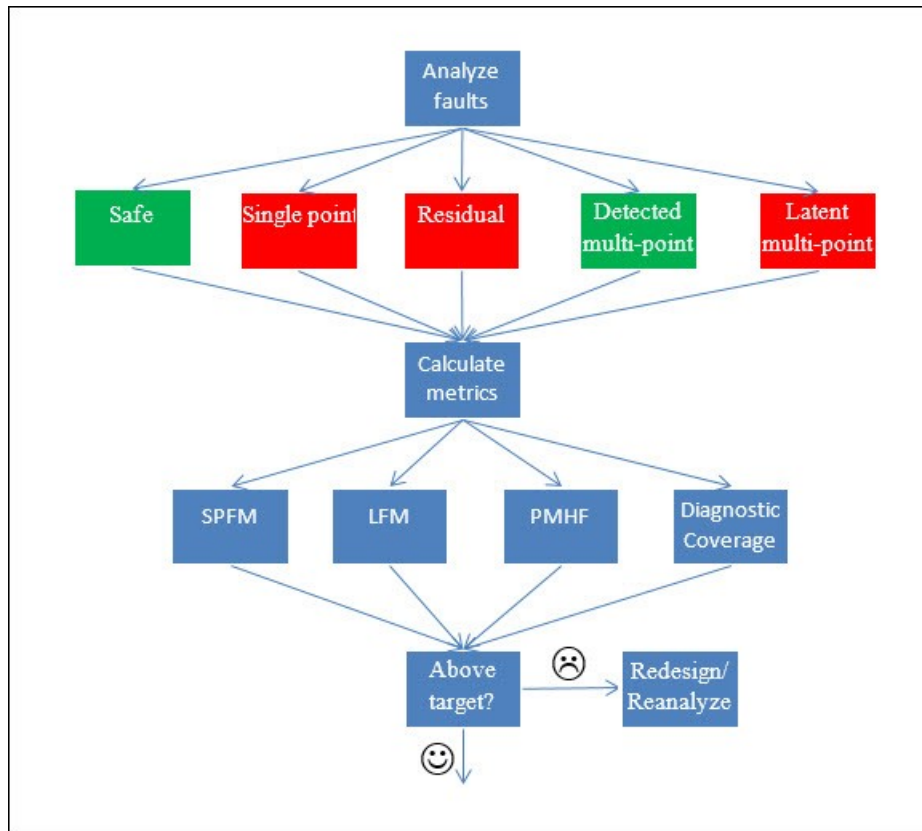
There are many sources for random hardware faults, from production process to extreme operating conditions, electronic interference and cosmic radiation. There are even some bad people trying to trigger them intentionally in the hope of getting secret keys. Each of those sources comes with a more or less accurate "fault model" which describes how faults from that source should be modeled at RTL or, more often, gate level and below. For example, electronic interference faults are modeled as two signals assuming the same value (referred to as "bridging"), and should be applied only to high frequency signals that lie within close proximity to one another after place and route. Production faults are modeled as gates getting stuck-at a given value forever and faults caused by cosmic radiation or due voltage noise are often modeled as random gates getting a wrong value for a cycle (referred to as soft faults or single event upset/transient), or at much lower probability, getting stuck at a wrong value forever. Since some of these sources produce faults that behave in a similar way, there is usually no need to test all of them. At RTL or gate level, most of the relevant faults can be modeled as gates assuming the wrong value for a cycle or getting stuck at a given value forever.

The various fault sources usually also come with some numbers that describe their probability of showing up under given conditions. The number of faults one can expect within a given chip depends obviously on the number of gates in the design, but also on other parameters such as production process and packaging, which might make some gates more vulnerable than others. This number can be determined by using scientific measurement documents such as [IEC 62380 \(https://ab-div-bdi-bl-blm.web.cern.ch/ab-div-bdi-bl-blm/RAMS/iec-tr-62380.e.pdf\)](https://ab-div-bdi-bl-blm.web.cern.ch/ab-div-bdi-bl-blm/RAMS/iec-tr-62380.e.pdf) for stuck-at errors and [JESD89 \(http://www.jedec.org/sites/default/files/docs/jesd89-2a.pdf\)](http://www.jedec.org/sites/default/files/docs/jesd89-2a.pdf) for soft errors, as detailed in section 10-A.3.4 of the ISO 26262 specification. Some of the numbers are contested so choosing the right source is important.

In order to determine the probability of a fault causing a safety critical failure ISO 26262 requires that they are analyzed and classified into six different bins: "safe", "single point", "residual", "detected multi-point", "perceived multi-point" and "latent multi-point". I'll keep the joy of some of the more gory details of this classification for future posts in the series, and try to explain each of those classifications as concisely as possible for now. "Safe" faults are faults that can't impact safety critical logic either because they lack physical connection, or they're masked by some logic along the way. "single point faults" are faults that can get to a safety critical logic, and when they get to it, there isn't any safety mechanism such as CRC, to detect or correct them. For ASIL C/D such faults are out of the question. "Residual" faults happen in an area that is buffered from safety critical functionality by some safety mechanism, but are still exactly like single point faults because, and this is where it starts to get funny, this safety mechanism can't catch them. Cynics might see this category as a tax break for ASIL C/D designs, allowing them to have

some “single point faults” without calling them that. Multi-point faults get the prize for the most confusing name, as they are actually faults that are detected or corrected by the safety mechanism. The reason why ISO prefers to look at the half glass empty and call them multi-point-faults is because in order for them to break anything, they would need another fault in the safety mechanism itself. “Detected multi-point” faults are the ones that are corrected & detected by the safety mechanism, “latent multi-point” faults are the ones that are corrected but there’s no indication they ever existed, and “perceived multi-point” faults are faults that are not detected, but have some noticeable impact on the driving experience. This category usually doesn’t apply to digital IC, as it is very unlikely that anything happening in a digital IC in the car will have a direct impact on the driver.

Determining the probability of a given fault to be of a specific type is usually done by taking a large enough sample of faults and measuring their distribution between the various types. Abstracting from details yet again, the end result of this process would be concrete numbers for “safe fault probability”, “single point fault probability”, “residual fault probability”, etc. With these numbers it is now possible to go ahead to the next step and calculate a few ISO 26262 metrics such as PMHF, SPFM, LFM and Diagnostic Coverage, the formulas for which are given in section 5 of the specification. The higher your ASIL, the higher you need to score on any of those metrics. If you’re above target then the smiling face at the bottom of the diagram below is most probably yours.



What happens if your hardware doesn’t score well enough for your ASIL? Well, there are two options you can take. The obvious one is to try to modify your hardware so that more faults fall into the good “safe” or “detected” bins. This could be done either by better decoupling non-safety-critical logic from safety-critical one, or by improving/adding safety mechanisms. We can call this option the “expensive” one, as it will have a significant cost not only in re-design, re-verification and time but also in gate count – as a rule of thumb, the stronger the safety mechanism, the more gates it will take. The less obvious alternative, is to improve the fault analysis and classification flow, so that more and more “worst case” assumptions which increase the “single”, “residual” or “latent” fault count, are replaced by realistic assumptions which increase the “safe” and “detected” bin count. This can boost your ISO 26262 metrics without modifying a single gate, and prevent overdesign and schedule delays. We shall go into great detail about this option, which we prefer to refer to as “smart” rather than “cheap”, in our [next article](https://verificationacademy.com/topics/planning-measurement-and-analysis/articles/ISO-26262-fault-analysis-worst-case-is-really-the-worst) (<https://verificationacademy.com/topics/planning-measurement-and-analysis/articles/ISO-26262-fault-analysis-worst-case-is-really-the-worst>) on this topic.

[Back to Top](#)

© Mentor, a Siemens Business, All rights reserved www.mentor.com
(<https://www.mentor.com/products/fv/>)

[Sitemap \(/sitemap\)](#)
[Terms & Conditions \(https://verificationacademy.com/terms-and-conditions\)](https://verificationacademy.com/terms-and-conditions)
[Verification Horizons Blog \(http://www.verificationhorizonsblog.com\)](http://www.verificationhorizonsblog.com)
[LinkedIn Group \(https://www.linkedin.com/groups/4668056\)](https://www.linkedin.com/groups/4668056)